Solution: Missing Element in Sorted Array

Problem Statement

Given an array `arr` containing `unique` integers in the `ascending` order, and integer `k`, return the **kth** missing number starting from the `arr[0]`.

The `missing numbers` are those that are not present in the array, and greater than `arr[0]`.

Examples

- **Example 1**:
  - **Input**: `arr = [3, 5, 6, 7], k = 2`
  - **Expected Output**: `8`
  - **Justification**: The missing numbers in sequence are 4 and 8. The 2nd missing number is 8.
- **Example 2**:
  - Input:** `arr = [1, 2, 4, 8], k = 3`
  - **Expected Output**: `6`
  - Justification:** The missing numbers are 3, 5, 6, etc. The 3rd missing number is 6.
- **Example 3**:
  - **Input**: `arr = [2, 3, 4, 7, 8, 10, 11], k = 2`
  - **Expected Output**: `6`
  - **Justification**: The missing numbers in this sequence start from 5, 6, 9... The 2nd missing number in this sequence is 6.

Solution

To solve this problem, we will use a binary search approach. This method is efficient because it significantly reduces the number of elements we need to inspect in the sorted array. By comparing the number of missing elements at the midpoint of the array with `k`, we can determine whether to search in the left or right half of the array.

This approach works effectively because the array is sorted, and we can calculate the number of missing elements up to any point. Since we're dealing with missing elements in a sequence, leveraging the sorted nature of the array allows us to quickly home in on the correct position, making this method both time-efficient and intuitive.

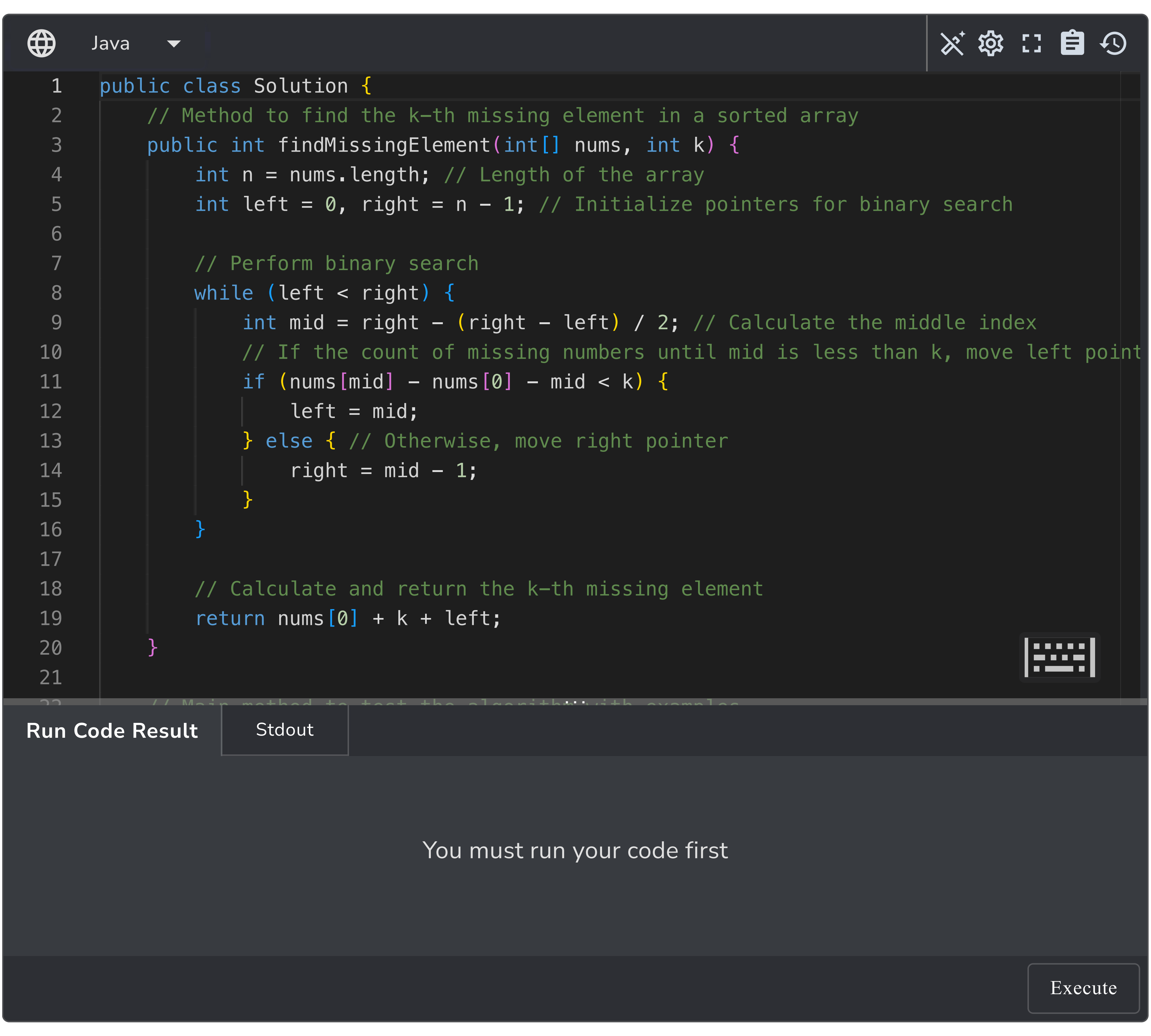
Step-by-step Algorithm

1. **Initialize Variables**:
   - Set `n` to the length of the input array `nums`.
   - Initialize two pointers `left = 0` and `right = n - 1` for binary search.
2. **Perform Binary Search**:
   - While `left` is less than `right`:
     - Calculate `mid` as `right - (right - left) / 2`. This finds the middle index between `left` and `right`.
     - Check if the number of missing elements up to `mid` is less than `k`.
       - This is done by `nums[mid] - nums[0] - mid`.
     - If the missing count is less than `k`, set `left` to `mid`. This moves the search to the right half of the current segment.
     - Otherwise, set `right` to `mid - 1`. This moves the search to the left half.
3. **Calculate and Return Result**:
   - After exiting the loop, calculate the k-th missing element as `nums[0] + k + left`.
   - Return this value.

Algorithm Walkthrough for Updated Example 3:

- **Input**: `arr = [2, 3, 4, 7, 8, 10, 11], k = 2`
- **Initialization**:
  - `n = 7` (length of array)
  - `left = 0`, `right = 6`
- **Binary Search Steps**:
  - **Iteration 1**:
    - `mid = 6 - (6 - 0) / 2 = 3`
    - Missing elements till `mid` = `arr[3] - arr[0] - (3 - 0) = 7 - 2 - 3 = 2`
    - Since 2 (missing count) is equal to `k`, move right pointer: `right = mid - 1 = 3 - 1 = 2`
  - **Iteration 2**:
    - `mid = 2 - (2 - 0) / 2 = 1`
    - Missing count = `arr[1] - arr[0] - (1 - 0) = 3 - 2 - 1 = 0`
    - Since 1 is less than `k`, move left pointer: `left = mid = 1`
  - **Iteration 3**:
    - `mid = 2 - (2 - 1) / 2 = 2 - 0 = 2`
    - Missing count = `arr[2] - arr[0] - (2 - 0) = 4 - 2 - 2 = 0`
    - Since 2 is less than `k`, move left pointer: `left = mid = 2`
- **Iteration 3**:
  - since left > right, `break` the loop.
- **Result Calculation**:
  - Exiting loop with `left = 2`
  - Calculate result: `nums[0] + k + left = 2 + 2 + 2 = 6`
- **Final Output**: `6.`

Code



```java
public class Solution {
    // Method to find the k-th missing element in a sorted array
    public int findMissingElement(int[] nums, int k) {
        int n = nums.length; // Length of the array
        int left = 0, right = n - 1; // Initialize pointers for binary search

        // Perform binary search
        while (left < right) {
            int mid = right - (right - left) / 2; // Calculate the middle index
            // If the count of missing numbers until mid is less than k, move left point
            if (nums[mid] - nums[0] - mid < k) {
                left = mid;
            } else { // Otherwise, move right pointer
                right = mid - 1;
            }
        }

        // Calculate and return the k-th missing element
        return nums[0] + k + left;
    }

```

Complexity Analysis

- **Time Complexity:** `O(log n)`
  - The algorithm uses binary search, dividing the array in half each step, leading to logarithmic time complexity.
- **Space Complexity:** `O(1)`
  - Only a constant amount of extra space is used in the algorithm, regardless of the input size.

← Previous
Missing Element In Sorted Array (Medium)

Next →
Number Of Substrings Containing All Three Characters (Medium)

☐ Mark as Completed